

Análisis de Cohortes - H&M

Este notebook realiza un análisis profesional de cohortes sobre el dataset de transacciones de H&M. El objetivo es entender la retención y el comportamiento de los clientes a lo largo del tiempo, identificando patrones clave para estrategias de fidelización.

```
# 1. Importación de librerías y configuración de Spark
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, min as spark_min, month, year,
datediff, countDistinct
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

spark = SparkSession.builder.appName("Cohortes_HYM").getOrCreate()
```

2. Carga y exploración inicial de datos

Cargamos el dataset de transacciones desde la carpeta `merge_pyspark` y exploramos la estructura básica.

```
# Cargar datos
df = spark.read.parquet('../merge_pyspark')
print(f"Total de registros: {df.count():,}")
df.printSchema()
df.show(5)

# 2.1. Estadísticas descriptivas iniciales
print("Clientes únicos:", df.select('customer_id').distinct().count())
print("Rango de fechas:", df.selectExpr("min(Fecha) as min_fecha",
"max(Fecha) as max_fecha").show())
print("Compras promedio por cliente:",
df.groupBy('customer_id').count().agg({'count': 'mean'}).show())

# 2.2. Distribución temporal de compras
compras_por_mes = df.groupBy(year('Fecha').alias('año'),
month('Fecha').alias('mes')).count().toPandas()
plt.figure(figsize=(10,5))
plt.plot(compras_por_mes['año'].astype(str) + '-' +
compras_por_mes['mes'].astype(str), compras_por_mes['count'],
marker='o')
plt.xticks(rotation=45)
plt.title('Distribución temporal de compras')
plt.xlabel('Mes')
```



```

+-----+-----+-----+-----+
+-----+-----+-----+-----+
|      customer_id|article_id|      Fecha|      price|
sales_channel_id|product_code|      prod_name|product_type_no|
product_type_name|product_group_name|graphical_appearance_no|
graphical_appearance_name|colour_group_code|colour_group_name|
perceived_colour_value_id|perceived_colour_value_name|
perceived_colour_master_id|perceived_colour_master_name|department_no|
department_name|index_code|      index_name|index_group_no|
index_group_name|section_no|      section_name|garment_group_no|
garment_group_name|      detail_desc|  FN|Active|
club_member_status|fashion_news_frequency|age|      postal_code|
+-----+-----+-----+-----+
+-----+-----+-----+-----+
+-----+-----+-----+-----+
+-----+-----+-----+-----+
+-----+-----+-----+-----+
+-----+-----+-----+-----+
+-----+-----+-----+-----+
+-----+-----+-----+-----+
+-----+-----+-----+-----+
+-----+-----+-----+-----+
+-----+-----+-----+-----+
+-----+-----+-----+-----+
|00030bc026428965d...| 673677015|2019-11-28|0.0254067796610169|
2|      673677|      Henry polo (1)|      252|      Sweater|
Garment Upper body|      1010016|      Solid|
73|      Dark Blue|      4|
Dark|      2|      Blue|
1616|      Knitwear|      A|      Ladieswear|      1|
Ladieswear|      11|      Womens Tailoring|      1003|
Knitwear|Jumper in a soft,...|NULL| NULL|      ACTIVE|
NONE| 57|2c29ae653a9282cce...|
|00030bc026428965d...| 673677010|2019-11-28|0.0254067796610169|
2|      673677|      Henry polo (1)|      252|      Sweater|
Garment Upper body|      1010016|      Solid|
43|      Dark Red|      4|
Dark|      18|      Red|
1616|      Knitwear|      A|      Ladieswear|      1|
Ladieswear|      11|      Womens Tailoring|      1003|
Knitwear|Jumper in a soft,...|NULL| NULL|      ACTIVE|
NONE| 57|2c29ae653a9282cce...|
|00083cda041544b2f...| 683001006|2019-02-10|0.0169322033898305|
2|      683001|      Pattern 7p Socks|      302|      Socks|
Socks & Tights|      1010001|      All over pattern|
10|      White|      3|
Light|      9|      White|
3611|Shopbasket Socks|      B|Lingeries/Tights|      1|
Ladieswear|      62|Womens Nightwear,...|      1021| Socks
and Tights|Fine-knit socks i...|NULL| NULL|      ACTIVE|
NONE| 29|24e3594738f327e8a...|

```

```
|00083cda041544b2f...| 568571002|2018-10-24|0.0135423728813559|
2|      568571|Spartacus cheeky ...|      59|  Swimwear bottom|
Swimwear|      1010016|      Solid|
9|      Black|      4|
Dark|      5|      Black|
4242|      Swimwear|      B|Lingeries/Tights|      1|
Ladieswear|      60|Womens Swimwear, ...|      1018|
Swimwear|Fully lined, text...|NULL| NULL|      ACTIVE|
NONE| 29|24e3594738f327e8a...|
|00083cda041544b2f...| 723469007|2019-02-10|0.0254067796610169|
2|      723469|Kelly Push (Melbo...|      306|      Bra|
Underwear|      1010016|      Solid|
8|      Dark Grey|      4|
Dark|      12|      Grey|
1334| Casual Lingerie|      B|Lingeries/Tights|      1|
Ladieswear|      61|      Womens Lingerie|      1017| Under-,
Nightwear|Push-up bras in s...|NULL| NULL|      ACTIVE|
NONE| 29|24e3594738f327e8a...|
+-----+-----+-----+-----+
+-----+-----+-----+-----+
+-----+-----+-----+-----+
+-----+-----+-----+-----+
+-----+-----+-----+-----+
+-----+-----+-----+-----+
+-----+-----+-----+-----+
+-----+-----+-----+-----+
+-----+-----+-----+-----+
+-----+-----+-----+-----+
+-----+-----+-----+-----+
only showing top 5 rows
```

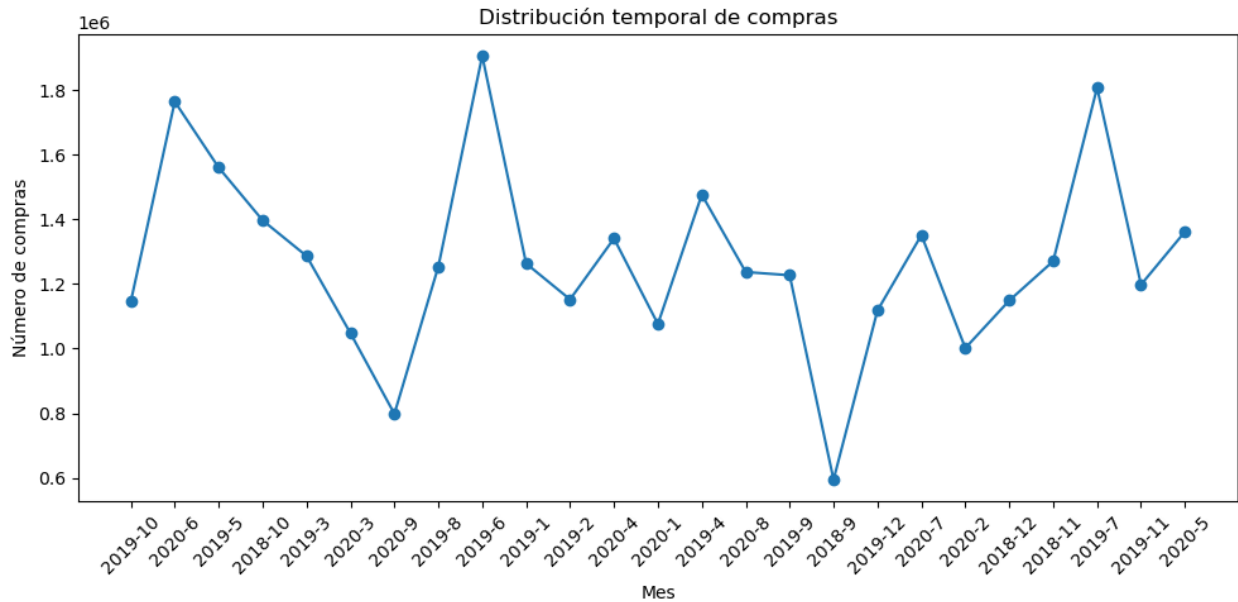
Clientes únicos: 1362281

```
+-----+
| min_fecha| max_fecha|
+-----+
|2018-09-20|2020-09-22|
+-----+
```

Rango de fechas: None

```
+-----+
|      avg(count)|
+-----+
|23.334630667241193|
+-----+
```

Compras promedio por cliente: None



3. Limpieza y preparación para análisis de cohortes

Seleccionamos las columnas relevantes y filtramos datos inválidos para asegurar la calidad del análisis.

```
# Filtrar datos válidos
df_clean = df.select('customer_id', 'Fecha', 'price') \
    .where(col('customer_id').isNotNull() & col('Fecha').isNotNull() &
    (col('price') > 0))
print(f"Registros limpios: {df_clean.count():,}")
```

Registros limpios: 31,788,324

4. Construcción de cohortes mensuales

Asignamos a cada cliente el mes de su primera compra, que será su cohorte. Calculamos el mes y año de cada transacción.

```
# Cohorte: mes y año de la primera compra
from pyspark.sql.functions import min as spark_min

cohort_df =
df_clean.groupBy('customer_id').agg(spark_min('Fecha').alias('cohort_date'))
cohort_df = cohort_df.withColumn('cohort_year', year('cohort_date'))
cohort_df = cohort_df.withColumn('cohort_month', month('cohort_date'))

# Unir cohorte al dataset
df_cohort = df_clean.join(cohort_df, on='customer_id')
```

```
df_cohort = df_cohort.withColumn('trans_year', year('Fecha'))
df_cohort = df_cohort.withColumn('trans_month', month('Fecha'))
```

5. Cálculo del índice de cohorte (meses desde primera compra)

Calculamos para cada transacción cuántos meses han pasado desde la cohorte del cliente.

```
# Índice de cohorte
df_cohort = df_cohort.withColumn(
    'cohort_index',
    (col('trans_year') - col('cohort_year')) * 12 +
    (col('trans_month') - col('cohort_month')) + 1
)
```

6. Matriz de retención de cohortes

Agrupamos por cohorte y mes para calcular la cantidad de clientes activos en cada período. Esto nos permite analizar la retención.

```
# Agrupar para matriz de retención
retention = df_cohort.groupBy('cohort_year', 'cohort_month',
    'cohort_index') \
    .agg(countDistinct('customer_id').alias('num_customers'))

# Convertir a pandas para visualización
retention_pd = retention.toPandas()
cohort_pivot = retention_pd.pivot_table(
    index=['cohort_year', 'cohort_month'],
    columns='cohort_index',
    values='num_customers'
)

# Normalizar para tasa de retención
cohort_size = cohort_pivot.iloc[:, 0]
retention_rate = cohort_pivot.divide(cohort_size, axis=0)
```

7. Visualización de la matriz de retención de cohortes

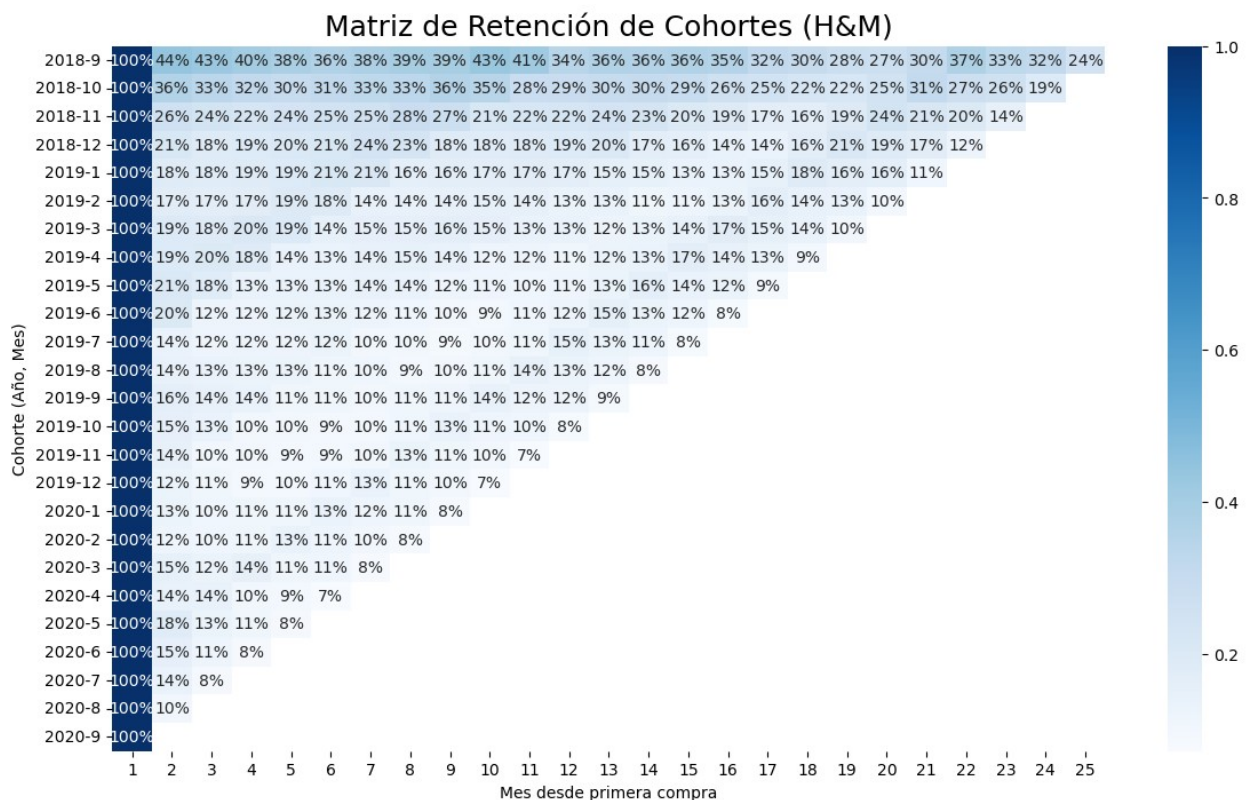
Graficamos la matriz de retención para visualizar cómo se comportan las cohortes a lo largo del tiempo.

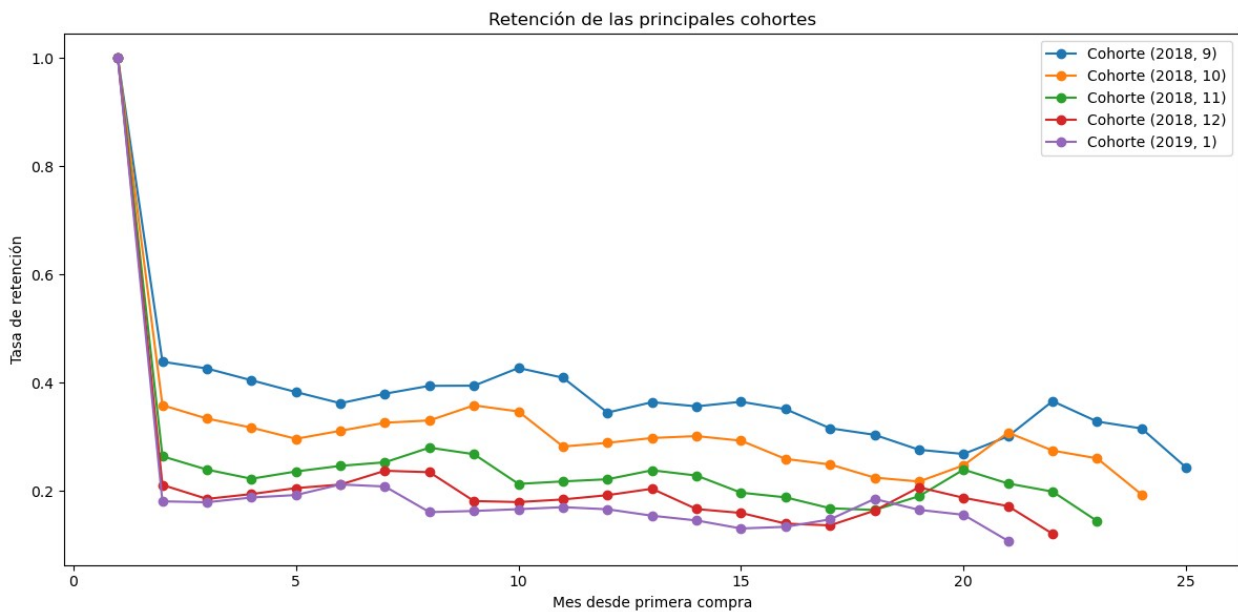
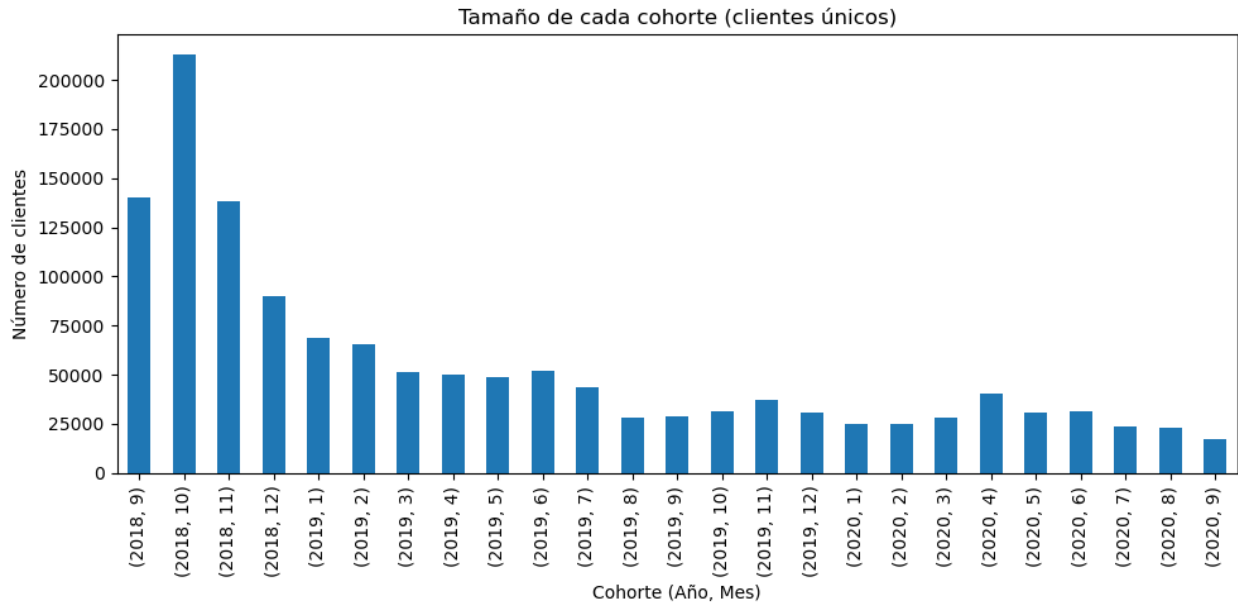
```
# Visualización de retención
plt.figure(figsize=(14, 8))
sns.heatmap(retention_rate, annot=True, fmt='.0%', cmap='Blues')
plt.title('Matriz de Retención de Cohortes (H&M)', fontsize=18)
plt.xlabel('Mes desde primera compra')
```

```
plt.ylabel('Cohorte (Año, Mes)')
plt.show()

# 7.1. Gráfico de tamaño de cohortes
plt.figure(figsize=(10,5))
cohort_size.plot(kind='bar')
plt.title('Tamaño de cada cohorte (clientes únicos)')
plt.xlabel('Cohorte (Año, Mes)')
plt.ylabel('Número de clientes')
plt.tight_layout()
plt.show()

# 7.2. Gráfico de retención de cohortes principales
plt.figure(figsize=(12,6))
for i in range(min(5, len(retention_rate))):
    plt.plot(retention_rate.columns, retention_rate.iloc[i],
marker='o', label=f'Cohorte {retention_rate.index[i]}')
plt.title('Retención de las principales cohortes')
plt.xlabel('Mes desde primera compra')
plt.ylabel('Tasa de retención')
plt.legend()
plt.tight_layout()
plt.show()
```





8. Análisis de cohortes y recomendaciones

Interpretamos los resultados y proponemos acciones para mejorar la retención según los patrones observados.

```
# Análisis descriptivo
print('Resumen de cohortes:')
print(f"Cohortes analizadas: {len(retention_rate)}")
print(f"Promedio de retención al mes 1: {retention_rate[1].mean():.2%}")
print(f"Promedio de retención al mes 3: {retention_rate[3].mean():.2%}")
```



```
{retention_rate[3].mean():.2%}")
if 6 in retention_rate.columns:
    promedio_mes_6 = f"{retention_rate[6].mean():.2%}"
else:
    promedio_mes_6 = "No disponible"
print(f"Promedio de retención al mes 6: {promedio_mes_6}")

print('''
Recomendaciones:
- Identificar cohortes con mayor caída de retención y analizar causas.
- Implementar campañas de fidelización en los primeros 3 meses.
- Personalizar comunicaciones para cohortes con mejor retención.
- Realizar seguimiento a cohortes nuevas para evaluar impacto de acciones.
''')

# 8.1. Explicación adicional
from IPython.display import Markdown
Markdown("""
**Interpretación de los gráficos:**

- El gráfico de barras muestra el tamaño de cada cohorte, útil para identificar meses con mayor captación de clientes.
- El heatmap permite visualizar la retención mes a mes, detectando rápidamente cohortes con mejor o peor desempeño.
- El gráfico de líneas compara la retención de las cohortes más grandes, ayudando a identificar patrones y oportunidades de mejora.

**Recomendaciones adicionales:**
- Analizar el impacto de campañas de marketing en cohortes específicas.
- Segmentar cohortes por ticket promedio o frecuencia de compra para estrategias personalizadas.
""")
```

Resumen de cohortes:

```
-----
-----
NameError                                Traceback (most recent call
last)
Cell In[1], line 3
      1 # Análisis descriptivo
      2 print('Resumen de cohortes:')
----> 3 print(f"Cohortes analizadas: {len(retention_rate)}")
      4 print(f"Promedio de retención al mes 1:
{retention_rate[1].mean():.2%}")
      5 print(f"Promedio de retención al mes 3:
{retention_rate[3].mean():.2%}")
```

```
NameError: name 'retention_rate' is not defined
```

9. Guardado de resultados y cierre de Spark

Exportamos la matriz de retención y cerramos la sesión de Spark.

```
# Guardar matriz de retención
retention_rate.to_csv('matriz_retencion_cohortes_hym.csv')
print('Matriz de retención exportada a
matriz_retencion_cohortes_hym.csv')

spark.stop()
```